Building a Capable Coding Agent for Resource-Constrained Environments

A critic-guided actor–controller approach (work in progress)

Yannick M. Bondo

Abstract. Most good coding agents today sit on top of a very large model behind an API. I wanted to know how far you can get without that: a small model, on a single modest machine, made into a real coding agent by the system built around it. The agent is an actor–controller–critic loop. A fine-tuned actor proposes typed actions under constrained decoding; a controller applies a hard safety veto and a scored selection rule; and a learned critic, trained offline with implicit Q-learning, is supposed to re-rank the actor's choices toward the ones that actually solve the task. The critic is the part I care most about and the part that is least finished. This note lays out the architecture and its objectives, shows with real examples what the agent is actually choosing between, and reports what I have learned so far. Three things have already changed the design. For a small model, reliability is cheaper to buy with structure than with prompting or fine-tuning (constrained decoding plus deterministic vetoes took invalid actions from 5 to 0 and the hardest suite from 7/21 to 11/21, with no training). A train/deploy prompt mismatch quietly hid a fine-tuning gain for longer than I would like to admit. And the hand-tuned controller currently drowns out both the actor and the critic. The critic does not help yet; that is the open problem, and the rest of the project is organized around it. Numbers here are snapshots of a moving system, not final benchmarks.

1. The Problem I Care About

A coding agent does what a developer does: read code, find the right place, make a change, run the tests, look at what broke, try again. The systems that do this well rely on the biggest models available, served from a data center. That is fine until you cannot use it. A company may need the agent to stay inside its own network because the code cannot leave the building. A developer may want something that runs on a laptop, offline, with no per-token bill. And smaller, local models are clearly where a lot of this is heading: a capable assistant on a modest box, eventually on a phone. What these share is a tight budget — limited memory and compute, and no giant model to lean on.

The obvious thing is to grab the largest model that fits and hope it is good enough. In my experience it is not. Small models are fluent but erratic. They will give you a sensible-looking next step and then, a beat later, do something plainly wrong: edit the test instead of the code, reference a file that does not exist, drop a secret straight into a config file. You cannot ship that. And you cannot fix it by asking the model nicely, because asking nicely is exactly what a small quantized model is bad at.

So I am trying something else. Push the capability out of the model and into the system around it. A small fine-tuned model proposes; a controller supplies the discipline the model cannot be trusted to supply on its own; and a learned critic, trained on the agent's own runs, supplies the judgment about which proposal will actually lead somewhere. If it works, the capability lives in the combination. This is a long project, and what follows is an honest progress report, including the parts that are not working.

2. The Agent: Actions, Loop, and Selection

The agent works over a small, typed set of actions:

$$\mathcal{A} = \{\, \texttt{search\_code}, \texttt{inspect\_file}, \texttt{edit\_file}, \texttt{run\_test}, \texttt{inspect\_diff}, \texttt{rollback\_edit}, \texttt{submit\_solution} \,\}. \tag{1}$$

Each action carries typed arguments: `inspect_file` needs a `path`, `search_code` needs a `query`, and so on. The vocabulary is small on purpose. A small, typed action space is what lets the controller check every step and enforce safety in code instead of hoping the model behaves.

At step t the runtime holds a state s_t: the task, the files seen and edited so far, recent actions and observations, and a little promoted memory. The actor is a policy $\pi_\phi(a \mid s)$, which is just the fine-tuned LLM; it returns a few candidate actions C_t. The controller filters those through a hard veto, scores what survives, runs the best one, and folds the result back into s_{t+1}. Figure 1 and Algorithm 1 give the shape of it.

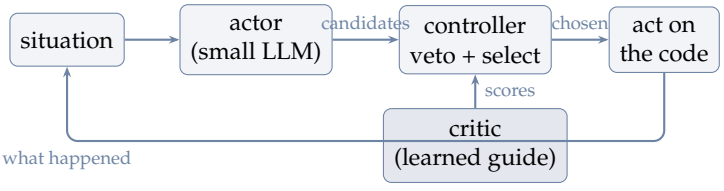

Figure 1: The loop. The actor proposes, the controller vetoes and selects, the critic is meant to inform the selection, and the result feeds back into the situation until the agent submits.

Algorithm 1 Controller step

Input: state s, allowed actions $\mathcal{A}_s \subseteq \mathcal{A}$, actor π_ϕ, critic (Q_θ, V_ψ) (optional)

1: $C \leftarrow \pi_\phi(\cdot \mid s, \mathcal{A}_s)$ ▷ k candidates, constrained decoding
2: $C \leftarrow \{\, c \in C : \textsc{Veto}(s, c) = \text{pass} \,\}$ ▷ hard safety filter
3: $C \leftarrow C \cup \textsc{Scaffold}(s, \mathcal{A}_s)$ ▷ rule-based fallback
4: **for all** $c \in C$ **do**
5: $\quad \text{score}(c) \leftarrow \sum_{\rho \in \Phi} w_\rho \mathbb{1}[\rho(s,c)] + w_\pi \mathbb{1}[c = c^\star_\pi] + w_Q \hat{z}\big(\hat{A}(s,c)\big)$
6: $c^\dagger \leftarrow \arg\max_c \text{score}(c)$; execute, observe, update s

Line 5 is where the real decision is made, so it is worth slowing down on. The first sum is a hand-tuned policy: a set Φ of about seventy situational rules ρ ("a regression is active, so prefer `rollback_edit`"), each adding a weight w_ρ. I split them into hard rails ($|w_\rho| \geq 5$, for safety and correctness) and soft nudges ($|w_\rho| < 5$, for taste and ordering). The second term rewards the actor's own top pick $c^\star_\pi$; the weight w_π is the knob that decides how much we trust the model over the table of rules. The third term is the critic: $\hat{A}(s,c)$ is the learned advantage of a candidate, z-normalized across the set and scaled by w_Q. The veto is not a weight. It is a filter that runs first, so no amount of score can buy back an unsafe action. One number I watch closely is the *model-agreement rate*: how often $c^\dagger$ equals $c^\star_\pi$. It tells me, bluntly, who is actually driving.

3. What the Agent Is Choosing Between

Before the machinery, it helps to see the actual material. Each evaluation case is a small, readable situation paired with the action a careful engineer would take. Here is one, with the model's real output underneath:

Case s27 (the model gets it wrong).
Situation: a dependency needs changing; the known file is `requirements.txt`.
Right move: `inspect_file` on `requirements.txt` before editing.
Model output: `{"action_type":"search_code", "arguments":{"query":"def parse_csv"}}`
Result: fail — it searched when it already knew the file to open.

Now almost the same surface behavior, but correct:

Case s30 (the model gets it right).
Situation: the relevant function lives in a very large file.
Right move: `search_code` for the symbol — do not read the whole huge file.
Model output: `{"action_type":"search_code", "arguments":{"query":"def parse_invoice_line"}}`
Result: pass.

That contrast is the whole difficulty in miniature. In both cases the model reaches for `search_code`. In one it is exactly right and in the other it is wrong, and the only thing that separates them is judgment about the situation. A safety rule cannot fix this, because searching is not unsafe; it is just sometimes the wrong call. This is the gap I am hoping a learned critic can close, and it is why so much of the note is about getting the rest of the system out of the way so judgment can matter.

4. Reliability by Construction

The first thing a small model needs is not to be smarter. It needs to be safe and well-formed by construction, because, again, you cannot talk it into either.

Constrained (guided) decoding. Let $\mathcal{G}(\mathcal{A}_s)$ be the set of JSON strings that satisfy the action schema: well-formed object, `action_type` in the allowed enum, the required arguments present. Rather than ask for valid output and check it after the fact, I constrain generation to that set. At each decoding step the token logits are masked to those that can still complete a valid string, so the model samples from

$$\pi_\phi(a \mid s) \;\propto\; p_\phi(a \mid s)\, \mathbb{1}\big[a \in \mathcal{G}(\mathcal{A}_s)\big], \tag{2}$$

and $\Pr[a \notin \mathcal{G}(\mathcal{A}_s)] = 0$. A malformed or disallowed action is no longer possible to emit. (A trap cost me an afternoon here: vLLM only honors the schema through its `json_schema` response format. If you also pass a plain `json_object` mode, it quietly wins and the schema is ignored.)

Deterministic vetoes. VETO rejects a short, enumerable list of catastrophes, no matter what the model wants: editing a protected test, acting on a path it has not actually seen, embedding a secret, submitting before any test has run. These are written as code, not requests.

Put together, safety and format stop being behaviors I hope for and become properties of the system. The effect was larger than I expected (Figure 2, left). On the hardest structured suite, constrained decoding erased the entire invalid-action failure class, five down to zero, and lifted pass-rate from 7/21 to 11/21. No training. That is the resource-constrained thesis in one experiment: spend engineering, not parameters, on the things engineering can guarantee, and save the model's limited capacity for judgment.

5. The Actor and Its Data

The actor is fine-tuned with QLoRA [2]: 4-bit base weights with small trained adapters, which is the only way the whole thing fits in the budget. The objective is ordinary supervised next-token loss over (situation $\rightarrow$ action) pairs, $\mathcal{L}(\phi) = -\sum \log \pi_\phi(a^\star \mid s)$, with $a^\star$ the reference action. Figure 3 shows it learning: the round-4 training loss falls steadily and held-out loss tracks it down, so this is real fitting rather than memorization of

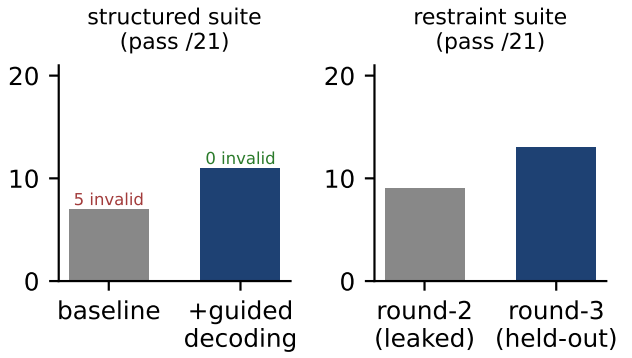

Figure 2: Two levers, real held-out numbers. **Left:** constraining the decoder removes invalid actions outright and lifts the hardest structured suite $7 \to 11$ (out of 21), with no training. **Right:** once the evaluation was de-leaked, a targeted fine-tuning round improved the restraint suite $9 \to 13$ on held-out cases. Structure and learning help in different places.

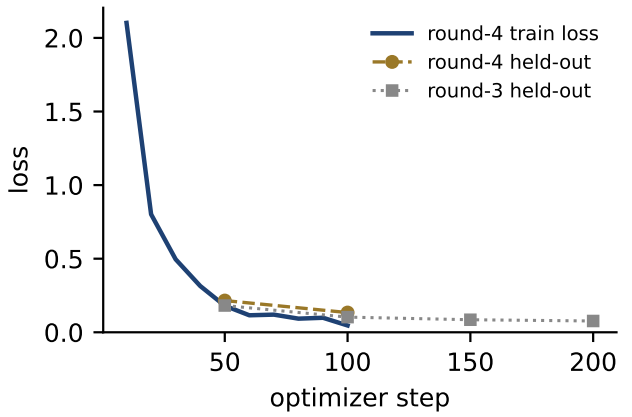

Figure 3: The actor learns. Round-4 training loss (solid) and held-out loss for rounds 3 and 4 all decrease; the held-out curves are what let me believe a gain is generalization, not memorization (see de-leaking below).

noise.

Where the data comes from. This is the part papers usually skip, and it matters more than the model. The corpus has three sources. *Scripted gold trajectories* are step-by-step solutions I hand-built for a ladder of seeded bugs, from a one-line fix up to a regression that has to be rolled back; they teach good sequences, not just good single steps. *Repair pairs* are mined from the agent's own failed runs — the wrong step it took, paired with the corrected one — so it learns to recover rather than only to perform in clean conditions. A *template bank* of parameterized synthetic cases fills in format and common patterns cheaply. The synthetic bulk gets down-weighted so it does not drown the judgment examples, which are the scarce and valuable part.

De-leaking, the boring lesson. An early version of the corpus contained the very scenarios I was testing on. The scores looked great. They were nonsense: the model had seen the test. Finding and removing that overlap is unglamorous and it is the difference between a number that means something and one that does not. Everything held-out in this note is post de-leak. On the cleaned data, a fine-tuning round aimed at the agent's weakest judgment cases moved the hardest restraint suite from 9/21 to 13/21 on held-out cases (Figure 2, right). That one I trust.

The bug that wasted a day: train/deploy prompt mismatch. For a while, fine-tuning looked like it did nothing. The actor kept choosing `search_code` on cases where it should have inspected a file it already knew about, exactly like case s27 above. I was ready to blame the data. The real cause was dumber and more useful: the model had been *trained* under one prompt format and *evaluated* under a different, fancier one. Shown its training-style prompt, the same checkpoint chose correctly on six of six probed cases. Shown the deployment prompt, zero of six. The skill was there the whole time; the wrapper hid it. (My first fix made it worse: swapping the deployment prompt to the bare training format recovered the judgment but threw away the safety guidance baked into the richer prompt, and one suite fell from 8/8 to 3/8 with the model happily editing protected tests again.) The right fix is to train on the deployment prompt, which I am doing now. The general lesson is that the prompt and the fine-tuning are one system and have to be designed together.

6. The Critic: The Part I Care About

The critic is the reason the project exists, and it is the least finished thing here. The idea is to learn, from the agent's own logged runs, how good each candidate action is, and let the controller use that to re-rank the actor toward choices that actually solve the task. I train it offline with implicit Q-learning [3] — from logs, no extra interaction — which is the right fit for a tight budget.

IQL fits a state value V_ψ and an action value Q_θ without ever asking about actions outside the data, by regressing V to an *expectile* of Q:

$$\mathcal{L}_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\Big[L_2^\tau\big(Q_{\bar\theta}(s,a) - V_\psi(s)\big)\Big], \qquad (3)$$

$$L_2^\tau(u) = \big|\tau - \mathbb{1}(u<0)\big|\,u^2,$$

$$\mathcal{L}_Q(\theta) = \mathbb{E}_{(s,a,s')\sim\mathcal{D}}\Big[\big(r + \gamma V_\psi(s') - Q_\theta(s,a)\big)^2\Big]. \qquad (4)$$

The expectile $\tau \in (0.5, 1)$ makes V a slightly optimistic estimate of the return that stays inside the data, and the controller then uses the advantage

$$\hat{A}(s,a) = Q_\theta(s,a) - V_\psi(s), \qquad (5)$$

z-normalized over the candidate set and weighted by

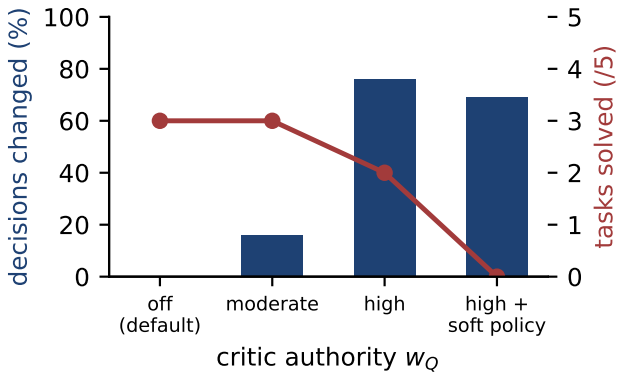

Figure 4: The critic today: invisible when quiet, harmful when loud. Bars are the fraction of decisions it changes as its weight w_Q rises; the line is tasks solved. More influence changes more decisions and solves fewer tasks, which says the bottleneck is value quality, not volume. (Early signal: a scripted actor and simulated execution over five trajectories; a real-actor, real-execution check is next.)

w_Q in Algorithm 1. The encoder is a small backbone with a shared LoRA trunk and separate Q and V heads, sized for the budget like everything else.

Now the honest part. **The critic does not help yet.** At the weight it ships with, its contribution ($\hat{z}\, w_Q \approx \pm 1.1$) is tiny next to the hand-tuned rules (± 5 to ± 8), so it never changes a single decision. It is inert. When I turn w_Q up far enough that it *can* change decisions, it does, and the results get worse, not better. Figure 4 is the whole story: as the critic gets more say it flips more of the agent's choices, and the number of tasks solved drops, from $3/5$ to $2/5$ to $0/5$ once I also soften the rule table. So the problem is not how loudly the critic speaks. It is that its value estimates are not good enough to re-rank by. That stung, but it is clarifying.

This is a result, not a wall, and it tells me where to dig. Because turning the critic up hurts, the work is the critic itself, not its weight: the experience it learns from (it likely needs far more, and clearer examples of bad actions and their consequences), the return definition and the IQL targets, and the encoder that turns a situation into something the value heads can use. Then I rerun the off-versus-on comparison on the real actor with real test execution, and I only keep the critic if it raises the number of tasks solved. That comparison is the experiment the whole project has been building toward.

7. Setup and How to Read the Numbers

The evaluation suites are hand-written situations like the ones in §3, probing four kinds of judgment: safety, restraint (do not edit before you understand), localization (find the right place), and stopping (submit when the tests pass instead of thrashing). They come in two surfaces: *raw*, free-form JSON over a chat prompt, and *structured*, the controller's own action interface. Because I tuned against the main suites while building the agent, I also wrote a separate, balanced set the system never saw, with half its cases rewarding "inspect the known file" and half rewarding the opposite reflex, so a gain cannot just be a one-sided bias. I report per-suite pass-rate, model-agreement rate (about 0.16 right now, which is the controller overriding the actor most of the time), and, for the critic, flip-rate.

On compute: everything — training, 4-bit serving, evaluation — runs on one modest GPU, on purpose. It stands in for the real target, a laptop or an on-prem box or eventually a phone. The point is not the specific card. The point is that nothing in the recipe secretly assumes a data center. And to say it once more, plainly: these numbers move week to week. Read them as where the system is, not as a final score.

Where This Is Going

What I have so far is a working loop, reliability bought with structure instead of scale, a fine-tuning gain I actually believe because the evaluation is clean, and a clear and slightly humbling picture of the hard part. The critic is not pulling its weight, and I finally understand why. The next stretch is making the learned guide actually guide: better experience to learn from, a cleaner training target, a stronger representation, and an honest before-and-after on real tasks. After that, I want to shrink the hand-tuned rule table down to just the safety rails and let the actor and critic make the rest of the calls, and to put the agent in front of a small slice of real GitHub issues as an external sanity check — not to chase a leaderboard a small model cannot win, but to show an ablated before-and-after of the pieces that are supposed to matter. I expect to be at this for a while. That is the point: it is a research program, and this note is a report from the middle of it.

References

[1] DeepSeek-AI. *DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence*. arXiv:2406.11931, 2024.

[2] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS, 2023.

[3] I. Kostrikov, A. Nair, S. Levine. *Offline Reinforcement Learning with Implicit Q-Learning*. ICLR, 2022.

[4] C. Jimenez, J. Yang, et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.